Redrob Candidate Ranker

India Runs: Data & AI Challenge

Team Name: Data Enthusiast

Team Leader: Praasuk Jain

Problem Statement: Recruiters miss perfect candidates because legacy systems rely on rigid keyword matching instead of understanding true fit and availability.

Solution Overview

What is your proposed solution?

A 4-stage intelligent pipeline that filters out honeypots, scores candidates using behavioral signals and BM25 keywords, and semantically re-ranks the top candidates using a local NLP embedding model.

What differentiates your approach?

Instead of just matching keywords, we evaluate a candidate's *actual availability* (via Redrob signals like response rate) and their *conceptual fit* (via dense vector embeddings of their summary), completely avoiding the trap of keyword-stuffing.

JD Understanding & Candidate Evaluation

What are the key requirements extracted?

Core competencies (embeddings, retrieval, python), product-company experience (vs pure service), and minimum 3 years of experience.

Which candidate signals are most important?

Beyond skills, we heavily weigh behavioral signals: `recruiter_response_rate`, `notice_period_days`, and `github_activity_score`. A candidate who matches perfectly but never responds is penalized, reflecting real-world recruiter priorities.

Ranking Methodology

How does your system retrieve, score, and rank?

1. **Hard Filter:** Drops candidates with `<3` YoE or honeypot traps.
2. **Scoring:** BM25 (keyword frequency) mixed with Behavioral multipliers.
3. **Re-Ranking:** The top 1000 are semantically scored against the JD.

What models/algorithms are used?

We use `sentence-transformers/all-MiniLM-L6-v2` for generating embeddings and Cosine Similarity to measure semantic distance.

How are multiple signals combined?

Final Score = (Semantic Score) + (BM25 Score) * (Behavioral Multiplier).

Explainability & Data Validation

How are ranking decisions explained?

Our pipeline dynamically generates a 1-2 sentence factual explanation for the top 100 candidates outlining exactly why they were ranked (e.g., their YoE, matching core skills, and strong response rate).

How do you prevent hallucinations?

No generative LLMs are used for the reasoning. The explanations are deterministically generated from verifiable data points within the candidate's JSON profile.

Handling inconsistent profiles?

We actively penalize "honeypots" — candidates who claim `expert` proficiency but have `0` duration of experience with that skill. We also filter out non-engineering titles (e.g., Marketing Manager).

End-to-End Workflow

1. **Ingest:** Stream `candidates.jsonl` (handles massive files efficiently without loading entirely into RAM).
2. **Filter & Evade:** Instantly discard unqualified profiles and honeypots.
3. **Retrieve:** Run BM25 algorithms to pull the Top 1,000 matches.
4. **Embed:** Convert candidate summaries into dense vectors using our local NLP model.
5. **Re-Rank:** Score via Cosine Similarity against the Job Description.
6. **Output:** Generate `team_PJ2001_IND.csv` with final rankings and generated reasoning.

System Architecture

- **Data Layer:** Streaming JSONL parser.
- **Filtering Engine:** Boolean logic rules and dictionary lookups.
- **Retrieval Engine:** Custom BM25 implementation for lightning-fast keyword relevance.
- **NLP Engine:** HuggingFace `sentence-transformers` running natively on CPU.
- **Output Layer:** CSV writer with tie-breaking stability sorting by `candidate_id`.

(Everything runs 100% locally. No external API calls are made.)

Results & Performance

What results demonstrate ranking quality?

The system successfully prioritized candidates who described building "recommendation engines" or "vector DBs" conceptually, even if they didn't explicitly list the exact keywords requested in the JD, proving semantic understanding.

How does it meet compute constraints?

- **Scale:** Successfully evaluated the entire 100,000 candidate dataset.
- **Speed:** Total end-to-end runtime of **17.62 seconds** on a standard CPU.
- **Memory:** Peaked well below the 16GB limit due to lazy-loading and stream processing.

Technologies Used

- **Python (3.11+):** Core pipeline execution.
- **Sentence-Transformers:** For generating semantic embeddings. Chosen because `all-MiniLM-L6-v2` is extremely fast on CPU while maintaining high contextual accuracy.
- **Scikit-Learn:** For computing Cosine Similarity arrays efficiently.
- **Streamlit:** For building the interactive sandbox web interface.
- **Hugging Face Spaces:** Used to host the live Sandbox environment.

Submission Assets

- **Live Sandbox Demo:** https://huggingface.co/spaces/praasukjain2001/redrob-ranker
- **GitHub Repository:** https://github.com/PJ2001-IND/redrob-ranker
- **Output File:** `team_PJ2001_IND.csv`
- **Metadata:** `submission_metadata.yaml`

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU